Helm Platform — Phase 1 Code Review

Reviewed by: Manus AI **Date:** 22 February 2026 **Commit Hash:** `c1a2b3d` (Assumed from latest pull)

---

1. Executive Summary

The Phase 1 code commit for the Helm platform backend is of **high quality**, demonstrating a strong architectural foundation and adherence to modern development practices. The FastAPI application is well-structured, containerised, and includes a solid baseline of tests. The developer has correctly interpreted the technical specification and laid the groundwork for all core features.

The most critical finding is that this commit **only contains the backend API**. The Flutter frontend is entirely absent and must be the focus of the next development phase.

Several areas contain placeholder logic and `TODO` comments, which is expected at this stage but requires clear prioritisation for the next development cycle. The overall assessment is **positive**, and the project is on a solid trajectory. This review outlines the key findings and provides specific, actionable recommendations for the developer to address before proceeding.

2. Overall Assessment

Area	Rating	Comments
Architectural Quality	✅ **Excellent**	Clean separation of concerns (models, schemas, routers). Use of Docker, Alembic, and Pydantic is best practice.
Code Quality & Readability	✅ **Excellent**	Code is clean, well-documented with docstrings, and follows Python best practices.
Specification Adherence	🟠 **Good**	All backend features from the spec are represented, but many are stubs. The frontend is missing.
Test Coverage	🟠 **Good**	Solid foundation with Pytest and an in-memory DB. Key success paths are covered, but gaps remain.
Security	🟠 **Good**	Core auth flow is present, but the development fallback for JWT validation is a risk that must be managed.

3. Critical Issues & Blockers

3.1. CRITICAL: Missing Frontend Application

- **Observation:** The repository contains **no frontend code**. The `flutter` directory and all related components, screens, and services are absent.
- **Impact:** This is a complete blocker for any user-facing testing or integration. The backend API, while functional, has no client.
- **Recommendation:** The **entire focus of the next development phase (Phase 2) must be the creation of the Flutter application**. The developer should follow the wireframes and full specification document to build out the core UI screens and connect them to the existing backend endpoints.

4. High-Priority Issues & Recommendations

4.1. HIGH: Incomplete Business Logic in API Endpoints

- **Observation:** Many endpoints contain placeholder logic, `TODO` comments, or return mock data instead of implementing the full business logic from the specification.
- **Impact:** Core features are not fully functional. This prevents end-to-end testing and delivery of key user value.
- **Recommendations:** The following `TODO` items must be prioritised for completion in the next backend-focused development phase:
  1. **`routers/loyalty.py`**: Implement the `get_current_multiplier` function to calculate the collective monthly spend from the `orders` table instead of returning a base rate.
  2. **`routers/loyalty.py`**: In `redeem_points`, add the validation logic to check an experience's point cost against the Strapi CMS before deducting points.
  3. **`routers/ai.py`**: Replace the fallback `_run_first_mate_agent` logic with the full LangChain agent orchestration, including the initialisation of domain-specialist tools (Pumps, Electrical, etc.) and the RAG pipeline connected to the `pgvector` database.
  4. **`routers/payments.py`**: While the Stripe Payment Intent creation is correct, the webhook handler needs to be made more robust with explicit logging and error handling for each event type.

4.2. HIGH: Insecure Authentication Fallback

- **Observation:** The `middleware/auth.py` file contains a fallback mechanism (`_decode_jwt_payload`) that decodes the JWT and extracts the user ID **without cryptographic verification** if the Supabase secret is not configured.
- **Impact:** If this code were ever to run in a misconfigured production environment, it would allow any validly-structured but unverified JWT to be accepted, completely bypassing authentication security.
- **Recommendation:**

1. Add an explicit check in `get_current_user_id` that raises a `RuntimeError` if the application environment (`APP_ENV`) is set to `production` but the Supabase secret is missing.
2. The development fallback should log a prominent warning on application startup to make it clear that authentication is insecure.

5. Medium & Low-Priority Recommendations

- **MEDIUM: Enhance Test Coverage:** While the test foundation is good, the developer should add tests for:
  - The loyalty points multiplier calculation logic (once implemented).
  - The Stripe webhook signature verification process.
  - Scenarios where a user attempts to access another user's resources (e.g., vessels, orders).
  - The AI agent's ability to correctly use vessel context.
- **LOW: Move Hardcoded Values to Configuration:**
  - The Helm Dash warehouse coordinates and pricing variables in `routers/helm_dash.py` should be moved into the `config.py` `Settings` class.
  - The default voyage checklist templates in `routers/checklists.py` are acceptable for now, but the long-term plan should be to manage these in the Strapi CMS for easier updates.
- **LOW: Frontend Directory Structure:** When the frontend is created, ensure it lives in a top-level `frontend/` directory, parallel to the `backend/` directory, to maintain a clean project structure.

6. Conclusion & Next Steps

The developer has delivered a high-quality backend foundation. The code is clean, scalable, and demonstrates a clear understanding of the project goals.

The immediate and sole priority is to begin development of the Flutter frontend.

Once the frontend is sufficiently developed to interact with the API, the next backend phase should focus on resolving the `TODO` items and implementing the full business logic outlined in the high-priority recommendations above.

This review serves as approval for the backend work in Phase 1, pending the critical addition of the frontend in Phase 2.